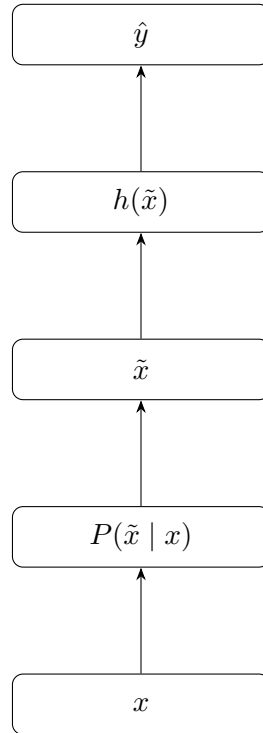


Adding Noise to the Input

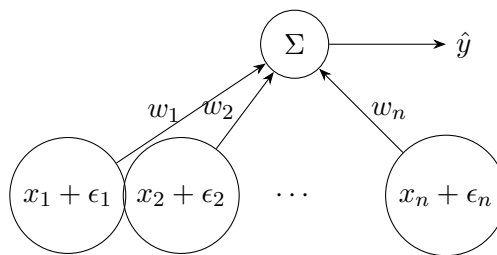
For a simple linear output neural network, adding Gaussian noise to the input is equivalent to adding an L_2 penalty (weight decay) on the weights.

It can also be interpreted as a form of data augmentation.

Input Noise Model



Single Output Neuron



Each input is corrupted by independent Gaussian noise,

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2),$$

therefore

$$\tilde{x}_i = x_i + \epsilon_i.$$

The clean network output is

$$\hat{y} = \sum_{i=1}^n w_i x_i.$$

The noisy output becomes

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i = \sum_{i=1}^n w_i (x_i + \epsilon_i).$$

Hence,

$$\tilde{y} = \hat{y} + \sum_{i=1}^n w_i \epsilon_i.$$

Expected Squared Error

We are interested in

$$E[(\tilde{y} - y)^2].$$

Substituting $\tilde{y}$,

$$E \left[\left((\hat{y} - y) + \sum_{i=1}^n w_i \epsilon_i \right)^2 \right].$$

Expanding,

$$= E[(\hat{y} - y)^2] + 2E \left[(\hat{y} - y) \sum_{i=1}^n w_i \epsilon_i \right] + E \left[\left(\sum_{i=1}^n w_i \epsilon_i \right)^2 \right].$$

Since

$$E[\epsilon_i] = 0,$$

and the noise is independent of $(\hat{y} - y)$,

$$E \left[(\hat{y} - y) \sum_i w_i \epsilon_i \right] = 0.$$

Thus,

$$E[(\tilde{y} - y)^2] = E[(\hat{y} - y)^2] + E \left[\left(\sum_{i=1}^n w_i \epsilon_i \right)^2 \right].$$

Because the noises are independent,

$$E[\epsilon_i \epsilon_j] = 0, \quad i \neq j.$$

Hence,

$$E \left[\left(\sum_{i=1}^n w_i \epsilon_i \right)^2 \right] = E \left[\sum_{i=1}^n w_i^2 \epsilon_i^2 \right].$$

Since

$$E[\epsilon_i^2] = \sigma^2,$$

we obtain

$$E[(\tilde{y} - y)^2] = E[(\hat{y} - y)^2] + \sigma^2 \sum_{i=1}^n w_i^2.$$

Conclusion

The objective function becomes

$$E[(\hat{y} - y)^2] + \sigma^2 \sum_{i=1}^n w_i^2$$

which is exactly the ordinary squared error plus an L_2 regularization (weight decay) term. Therefore,

$$\text{Adding Gaussian input noise} \iff \text{Weight Decay (} L_2 \text{ Regularization)}$$

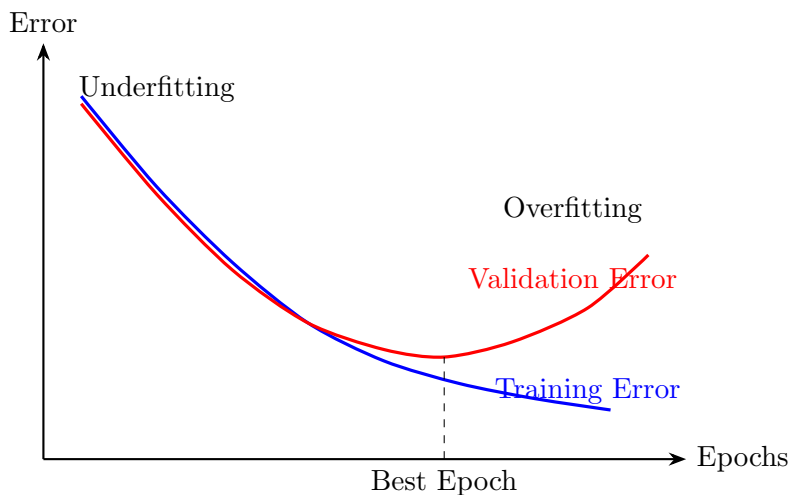
1 Early Stopping

When training a machine learning model, we usually monitor two quantities:

- **Training Error:** Measures how well the model fits the training data.
- **Validation Error:** Measures how well the trained model generalizes to unseen data.

The trade-off between these two errors determines whether the model is underfitting or overfitting.

- If the model is trained for too few epochs, both the training and validation errors remain high, indicating **underfitting**.
- If the model is trained for too many epochs, the training error continues to decrease while the validation error begins to increase, indicating **overfitting**.



1.1 Why Early Stopping?

The objective of early stopping is to stop training before the model starts overfitting.

The best model is usually the one corresponding to the minimum validation error rather than the minimum training error.

2 Patience

In practice, especially in deep learning, validation error may fluctuate slightly from one epoch to another.

Therefore, instead of stopping immediately when validation error increases, we introduce a parameter called **patience**.

Patience = Number of epochs to wait after the last improvement in validation error.

2.1 Example

Suppose

$$\text{Patience} = 5.$$

If the validation error does not improve for five consecutive epochs, training stops even if the training error is still decreasing.

2.2 Why Patience Matters

- **Patience Too Small**

Training may stop too early due to temporary fluctuations in validation error.

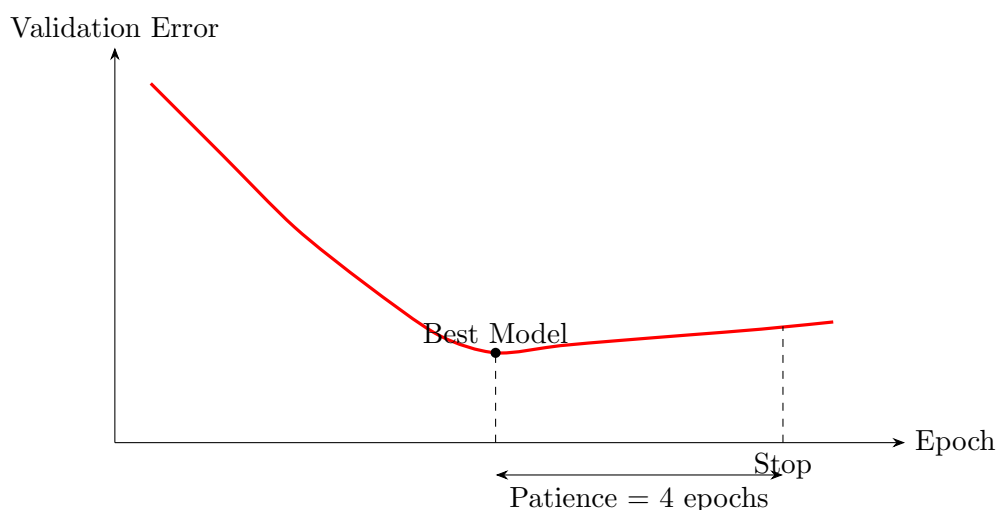
The resulting model may still be underfitting.

- **Patience Too Large**

Training continues for many unnecessary epochs.

Although the training error keeps decreasing, the validation error may continue increasing, resulting in overfitting.

3 Illustration of Patience



4 Algorithm

Suppose

- Current epoch = k
- Patience = P

If there has been no improvement in validation error during the previous P epochs, then

Stop Training and Restore the Model
saved at epoch $(k - P)$.

Key Takeaways

- Training error continuously decreases during optimization.
- Validation error initially decreases but eventually increases due to overfitting.
- Early stopping selects the model corresponding to the minimum validation error.
- Patience prevents stopping because of temporary fluctuations in validation performance.
- A suitable patience value improves generalization while avoiding unnecessary training.

5 Early Stopping as Regularization

Early stopping is one of the simplest and most effective forms of regularization in deep learning.

- It is one of the most widely used regularization techniques.
- It can be used independently or together with other regularizers such as L_2 regularization, dropout, etc.
- It prevents the model parameters from growing excessively, thereby reducing overfitting.

5.1 Intuition

Recall the SGD update rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t),$$

or equivalently,

$$\mathbf{w}_t = \mathbf{w}_0 - \eta \sum_{i=1}^t \nabla L(\mathbf{w}_{i-1}).$$

Suppose

$$\|\nabla L(\mathbf{w})\| \leq \tau,$$

where τ is the maximum magnitude of the gradient.

Then,

$$\|\mathbf{w}_t\| \leq \|\mathbf{w}_0\| + \eta t \tau.$$

Hence, the magnitude of the parameter vector can grow at most linearly with the number of iterations.

Therefore, stopping training early prevents the weights from becoming unnecessarily large.

5.2 Quadratic Approximation of the Loss

Assume the loss is locally quadratic around its optimum $\mathbf{w}^*$.

Using the second-order Taylor expansion,

$$L(\mathbf{w}) = L(\mathbf{w}^*) + (\mathbf{w} - \mathbf{w}^*)^T \nabla L(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T H(\mathbf{w} - \mathbf{w}^*),$$

where

$$H = \nabla^2 L(\mathbf{w}^*)$$

is the Hessian matrix.

Since

$$\nabla L(\mathbf{w}^*) = 0,$$

the approximation becomes

$$L(\mathbf{w}) = L(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T H(\mathbf{w} - \mathbf{w}^*).$$

Differentiating,

$$\nabla L(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*).$$

5.3 SGD Update

Substituting the gradient into SGD,

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta H(\mathbf{w}_{t-1} - \mathbf{w}^*).$$

Rearranging,

$$\boxed{\mathbf{w}_t = (I - \eta H)\mathbf{w}_{t-1} + \eta H\mathbf{w}^*}.$$

5.4 Eigenvalue Decomposition

Let the Hessian have the eigendecomposition

$$H = Q\Lambda Q^T,$$

where

- Q contains the orthonormal eigenvectors,
- $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

Then,

$$\boxed{\mathbf{w}_t = (I - \eta Q\Lambda Q^T)\mathbf{w}_{t-1} + \eta Q\Lambda Q^T\mathbf{w}^*}.$$

5.5 Solution

Assume the initialization

$$\mathbf{w}_0 = \mathbf{0}.$$

Then,

$$\boxed{\mathbf{w}_t = Q [I - (I - \eta\Lambda)^t] Q^T \mathbf{w}^*}.$$

5.6 Connection with L_2 Regularization

Recall the solution obtained using L_2 regularization,

$$\tilde{\mathbf{w}} = Q [I - (\Lambda + \alpha I)^{-1} \alpha] Q^T \mathbf{w}^*.$$

Comparing the two expressions,

$$(I - \eta \Lambda)^t \approx (\Lambda + \alpha I)^{-1} \alpha,$$

shows that early stopping behaves similarly to L_2 regularization.

Thus, early stopping is often viewed as an implicit form of regularization.

6 Why Does Early Stopping Regularize?

Early stopping allows only a limited number of parameter updates.

Suppose a parameter w_i corresponds to an important direction of the loss surface.

Then,

$$\left| \frac{\partial L}{\partial w_i} \right|$$

is relatively large, so the parameter receives larger updates and converges quickly.

On the other hand, if

$$\left| \frac{\partial L}{\partial w_i} \right|$$

is small, then the parameter changes only slowly.

Since training is stopped before complete convergence, these parameters never become excessively large.

Consequently,

- parameters corresponding to important directions are learned,
- parameters corresponding to less important directions remain small,
- model complexity is reduced,
- overfitting is minimized.

Key Takeaways

1. Early stopping is one of the most effective implicit regularization techniques.
2. It limits the growth of model parameters by restricting the number of SGD updates.
3. Under a quadratic approximation of the loss, early stopping behaves similarly to L_2 regularization.
4. Important parameters converge rapidly, whereas less important parameters remain small.
5. The resulting model usually generalizes better to unseen data.

7 Ensemble Methods

An **Ensemble Method** combines the predictions of multiple machine learning models to obtain a more accurate and robust final prediction.

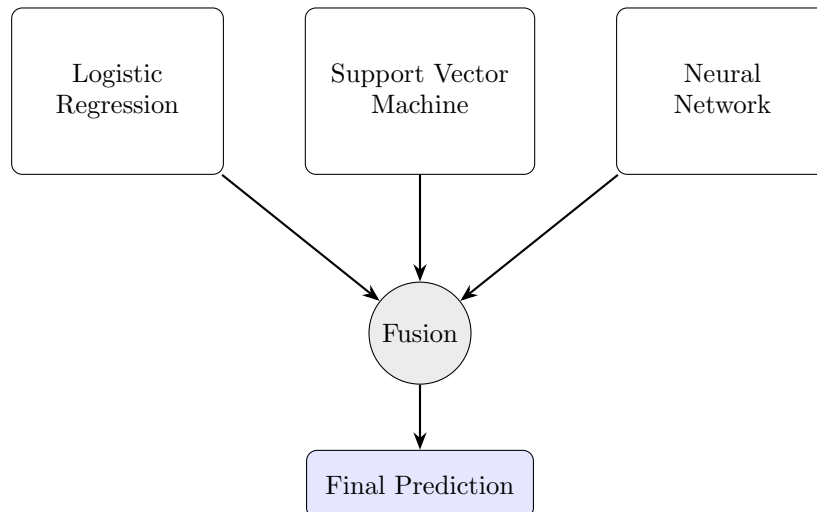
The individual models can be

- Different learning algorithms (Logistic Regression, SVM, Decision Tree, Neural Network, etc.)
- Different instances of the same learning algorithm.

The individual models are typically trained using

- Different hyperparameters,
- Different subsets of features,
- Different subsets of training samples.

7.1 Architecture



The outputs of all base learners are combined using an aggregation rule such as

Average, Majority Voting, Weighted Average.

8 Bagging

Bagging stands for

Bootstrap Aggregating

It is an ensemble technique in which

1. Multiple bootstrap datasets are created from the original training data.
2. A separate classifier is trained on each bootstrap sample.
3. The predictions of all classifiers are combined to obtain the final prediction.

8.1 Why Does Bagging Work?

Suppose there are k classifiers.

Each classifier produces an error

$$e_i.$$

Assume

$$E[e_i] = 0,$$

and

$$\text{Var}(e_i) = V.$$

Further,

$$\text{Cov}(e_i, e_j) = C, \quad i \neq j.$$

8.2 Average Prediction Error

The ensemble prediction error is

$$\bar{e} = \frac{1}{k} \sum_{i=1}^k e_i.$$

The mean squared error is

$$MSE = E[\bar{e}^2].$$

Substituting,

$$= E \left[\left(\frac{1}{k} \sum_{i=1}^k e_i \right)^2 \right].$$

Expanding,

$$= \frac{1}{k^2} E \left[\sum_i e_i^2 + \sum_{i \neq j} e_i e_j \right].$$

Using

$$E[e_i^2] = V,$$

and

$$E[e_i e_j] = C,$$

we obtain

$$= \frac{1}{k^2} [kV + k(k-1)C].$$

Hence,

$$\boxed{MSE = \frac{V}{k} + \frac{k-1}{k}C.}$$

9 Interpretation

The ensemble error depends on two quantities:

1. Variance of individual models (V)
2. Correlation between their errors (C)

Case 1: Independent Errors

If the individual models are independent,

$$C = 0,$$

then

$$MSE = \frac{V}{k}.$$

Therefore,

Increasing the number of models reduces the error.

Case 2: Perfectly Correlated Errors

If every model makes exactly the same mistakes,

$$C = V,$$

then

$$MSE = \frac{V}{k} + \frac{k-1}{k}V = V.$$

Hence,

No improvement is obtained by bagging.

Conclusion

Bagging is effective only when the individual models make different (uncorrelated) errors.

Therefore,

Lower correlation among models $\implies$ Greater improvement from ensemble learning.

Key Points

- Ensemble methods combine multiple models.
- Diversity among models is essential.
- Bagging reduces variance.
- Independent errors provide maximum reduction in prediction error.
- Highly correlated models offer little benefit.